

FITBUDDY

AI-Powered Personalized Workout and Wellness Assistant

PHASE 05

Project Development Phase

Record the implementation of FitBuddy and the final source-file organization used for the working project.

Prepared for phase-wise submission
30 September 2026

Phase objective: Record the implementation of FitBuddy and the final source-file organization used for the working project.

1. Development Stack

- Python application using FastAPI and Uvicorn.
- Jinja2 for server-rendered HTML pages.
- SQLAlchemy with SQLite for persistence.
- Pydantic and Pydantic Settings for validation/configuration.
- Google GenAI Python SDK for live Gemini integration.
- Pytest and TestClient for automated testing.

2. Final Source Organization

- app/: core application package.
- app/services/: AI client and generation/revision services.
- templates/: base, home, result, users, and error pages.
- static/: CSS and JavaScript.
- tests/: fixture configuration and application tests.
- data/: SQLite runtime location.
- .vscode/: debug configuration.
- .env.example / requirements.txt / README.md: configuration, dependencies, and setup documentation.

3. Development Flow

- Step 1: create and activate .venv.
- Step 2: install requirements from requirements.txt.
- Step 3: copy .env.example to .env.
- Step 4: keep MOCK_AI=true for first local verification.
- Step 5: run pytest -q.
- Step 6: start uvicorn app.main:app --reload.
- Step 7: use the browser at http://127.0.0.1:8000.
- Step 8: optionally configure live Gemini access through .env after the mock flow is confirmed.

4. Implementation Highlights

- Routes validate form inputs and coordinate generation plus persistence.
- The first plan is saved as original_plan; feedback revisions are stored in updated_plan.
- The AI client centralizes structured JSON generation and converts SDK failures into GeminiServiceError.
- Mock implementations mirror the expected schemas, enabling development without external AI access.
- FastAPI startup initializes database tables automatically.

5. Safety and Maintainability

- Prompts explicitly avoid diagnosis, treatment, medication, extreme dieting, and unsafe challenges.
- Generated content includes a safety note.
- The project separates configuration, persistence, validation, routing, UI, and AI logic to simplify future maintenance.
- Before public deployment, authentication/authorization should be added to the local users/admin area.

Key Files and Responsibilities

File	Responsibility
app/main.py	FastAPI app + startup + static files
app/routes.py	HTML routes + form handling + persistence flow
app/schemas.py	Input/output validation models
app/models.py	SQLAlchemy User/Plan models
app/database.py	Engine, sessions, table initialization
app/services/gemini_client.py	Gemini client + structured generation
app/services/gemini_generator.py	7-day workout generation
app/services/gemini_flash_generator.py	Nutrition/recovery generation
app/services/updated_plan.py	Feedback-based plan revision
tests/test_app.py	Main automated flow tests

Phase Deliverables / Evidence

- Implemented source structure
- Development and configuration flow
- Key file responsibilities

Submission Note

This phase document is intended to be submitted together with the corresponding FitBuddy source project and evidence captured during execution. Keep API keys, personal credentials, and other secrets out of screenshots and submissions.